POSTS AND TELECOMMUNICATIONS INSTITUTE OF TECHNOLOGY



INFORMATION SYSTEM ANALYSIS AND DESIGN

Lecturer : **Tran Dinh Que**

Student Name : **Hoàng Gia Khánh - B23DCDT140**

Class : **E23TTNT01**

Nhóm : **04**

Hà Nội – 2026

1. Task 1

1.1 System description

The Multimodal Search System for E-Commerce is a prototype online shopping search platform that lets customers find products and orders through three interaction modes: typed keywords, spoken queries, and uploaded product images. Regardless of the input type, the system converts each query into a common query representation, retrieves candidate items from the data layer, ranks them, and returns the results to the customer.

The system follows a three-layer architecture. The Presentation Layer collects user input and displays results. The Application/Intelligence Layer interprets queries, converts modalities, retrieves candidates, and ranks them. The Data Layer stores products, orders, images, and vector embeddings. The Presentation Layer never accesses the data directly, which keeps the system maintainable and extensible.

The prototype is not a production system. Its purpose is to show how system design, UML modeling, and a simple AI-oriented implementation fit together.

1.2 Actors analysis:

Actor	**Type**	**Direct Interaction**	**Justification**
Customer	Human, external	Searches products by keyword, voice, or image; searches orders by ID; views product and order details	Primary user who initiates every search and consumes the results
Speech Recognition Component	External system (simulated)	Receives the voice input and returns the transcribed text	The voice pipeline needs speech-to-text; the prototype simulates it by passing transcribed text

Developer / Administrator	Human, internal	Maintains the product dataset, product images, and embeddings	Keeps the searchable data current; has no use case in the diagram, so it is optional

1.3 Functional requirements

ID	Category	Requirement	Priority
FR01	Text input	The system shall accept a keyword or natural-language text query from the customer.	Must
FR02	Text processing	The system shall normalize text queries (lowercasing, tokenization) before retrieval.	Must
FR03	Voice input	The system shall accept a voice query and convert it to text through the SpeechService (simulated by passing transcribed text).	Must
FR04	Image input	The system shall accept an uploaded product image and convert it into a feature vector.	Must
FR05	Query representation	The system shall convert text, voice, and image inputs into a common query object through the QueryService.	Must
FR06	Text retrieval	The system shall retrieve products whose name, category, or color matches the query keywords.	Must
FR07	Image retrieval	The system shall retrieve products by cosine similarity between the query embedding and product embeddings.	Must
FR08	Ranking	The system shall rank retrieved candidates by relevance score in descending order, as a step separate from retrieval.	Must
FR09	Result display	The system shall display each result with product name, category, color, price, stock, and ranking score.	Must

FR10	Order search	The system shall find an order by order ID and return its status and total.	Must
FR11	Empty results	The system shall display a "no results found" message when no product or order matches.	Should
FR12	Product view	The system shall display the details and availability of a selected product.	Should
FR13	Order view	The system shall display the details of a selected order.	Should
FR14	Multimodal fusion	The system shall combine text and image (and optionally voice) scores using configurable weights ($\lambda t + \lambda i + \lambda v = 1$).	Should
FR15	Filtering	The system shall filter results by maximum price and by category.	Could
FR16	Stock-aware ranking	The system shall rank in-stock products above out-of-stock products with similar relevance.	Could
FR17	Dataset	The system shall provide a product dataset of at least 10 products, each with an embedding and an image.	Must

1.4 Non-functional requirements

ID	Category	Requirement
NFR01	Performance	95% of text and voice searches return results in under 1 second.
NFR02	Performance	95% of image searches return results in under 2 seconds.
NFR03	Maintainability	The Presentation Layer shall have no direct dependency on the Data Layer; all access goes through the Application Layer (UI → Application → Data).
NFR04	Maintainability	Each service (Query, Speech, Image, Search, Ranking) shall expose a documented interface with docstrings.

NFR05	Extensibility	The simulated SpeechService and artificial embeddings shall be replaceable with real speech-to-text and a real image model without changing other modules.
NFR06	Usability	Results shall be shown in ranked order with scores, so a user can understand why a product appears first.
NFR07	Reliability	Invalid input (empty query, unknown order ID, zero-norm vector) shall not crash the program; it shall return an empty result or a message.
NFR08	Data integrity	Product IDs and order IDs shall be unique.
NFR09	Portability	The prototype shall run on Python 3.8+ on Windows, Linux, and macOS, with NumPy as the only required package.
NFR10	Testability	The prototype shall be evaluated on a test set of at least 10 queries covering all three modes, with a target success rate of at least 75%.
NFR11	Security (scope note)	Authentication and encryption are out of scope for this prototype and documented as a limitation.

1.5 input modalities

Modality	Input	Processing
Text	Keyword or natural-language query (ex: "black running shoes")	Text normalization (lowercasing, tokenization), then keyword-based retrieval
Voice	Spoken query (simulated by providing the transcribed text)	Speech-to-text, then the same pipeline as text search
Image	Product photograph uploaded by the customer	Image encoder produces a feature vector, then cosine-similarity search against product embeddings
Multimodal (extension)	Text + image (+ optional voice) together	Weighted fusion of representations and scores

1.6 search outputs

Product search returns a ranked list of matching products. Each result contains:

- product ID and name
- category, color, price, and stock level
- a relevance score (keyword match score for text and voice; cosine similarity for image; combined score for multimodal queries)

Results are ordered from highest to lowest score, with the top-ranked product shown first.

Order search returns the matching order's details:

- order ID
- status (Shipped, Delivered)
- total amount

No-match case: If no product or order matches the query, the system returns an empty result with a "no results found" message.

2. Task 2 : Usecase Diagram

2.1 Overview

The use case diagram shows how the customer interacts with the multimodal search system. The system boundary contains all search and view functions. The Customer is the primary actor

2.3 Use Cases

Use Case	Description
Search Product	General product search. It is specialized by the three search modes below.
Search by Keyword	Finds products using a typed text query.
Search by Voice	Finds products using a spoken query that is converted to text.
Search by Image	Finds visually similar products using an uploaded product photo.

Search Order	Finds an order by its order ID.
View Product	Displays the details and availability of a selected product.
View Order	Displays the status and total of a selected order.

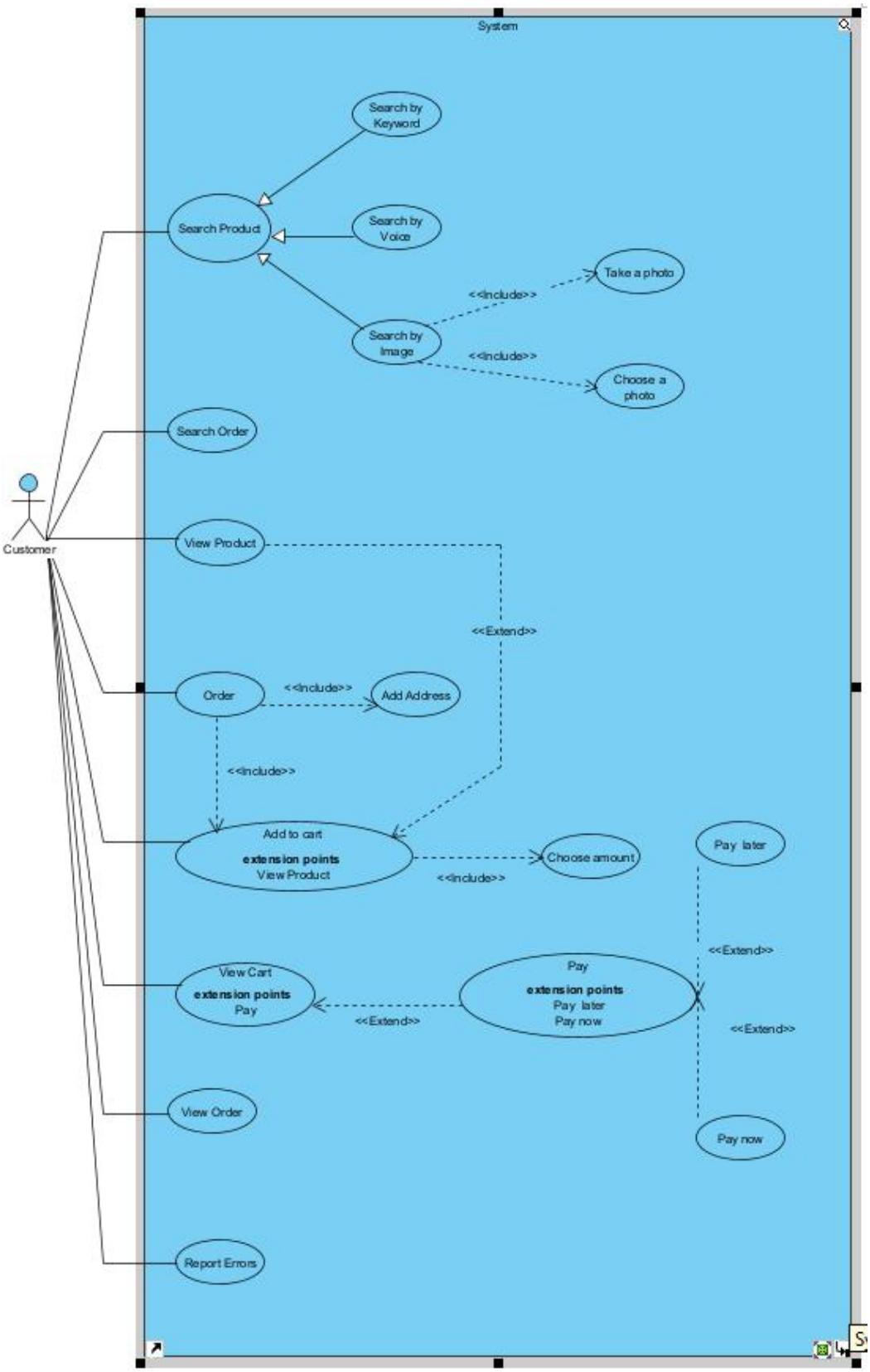


Figure 1 : Usecase Diagram

3. Task 3: Three-Layer Architecture

3.1 Overview

The system follows a three-layer architecture that separates user interaction, search logic, and data storage. Each layer has one responsibility and depends only on the layer directly below it:

Presentation Layer → Application / Intelligence Layer → Data Layer

This separation lets each layer be changed or replaced without affecting the others. For example, the simulated speech recognition and the artificial image embeddings can later be replaced by real models without changing the user interface or the database.

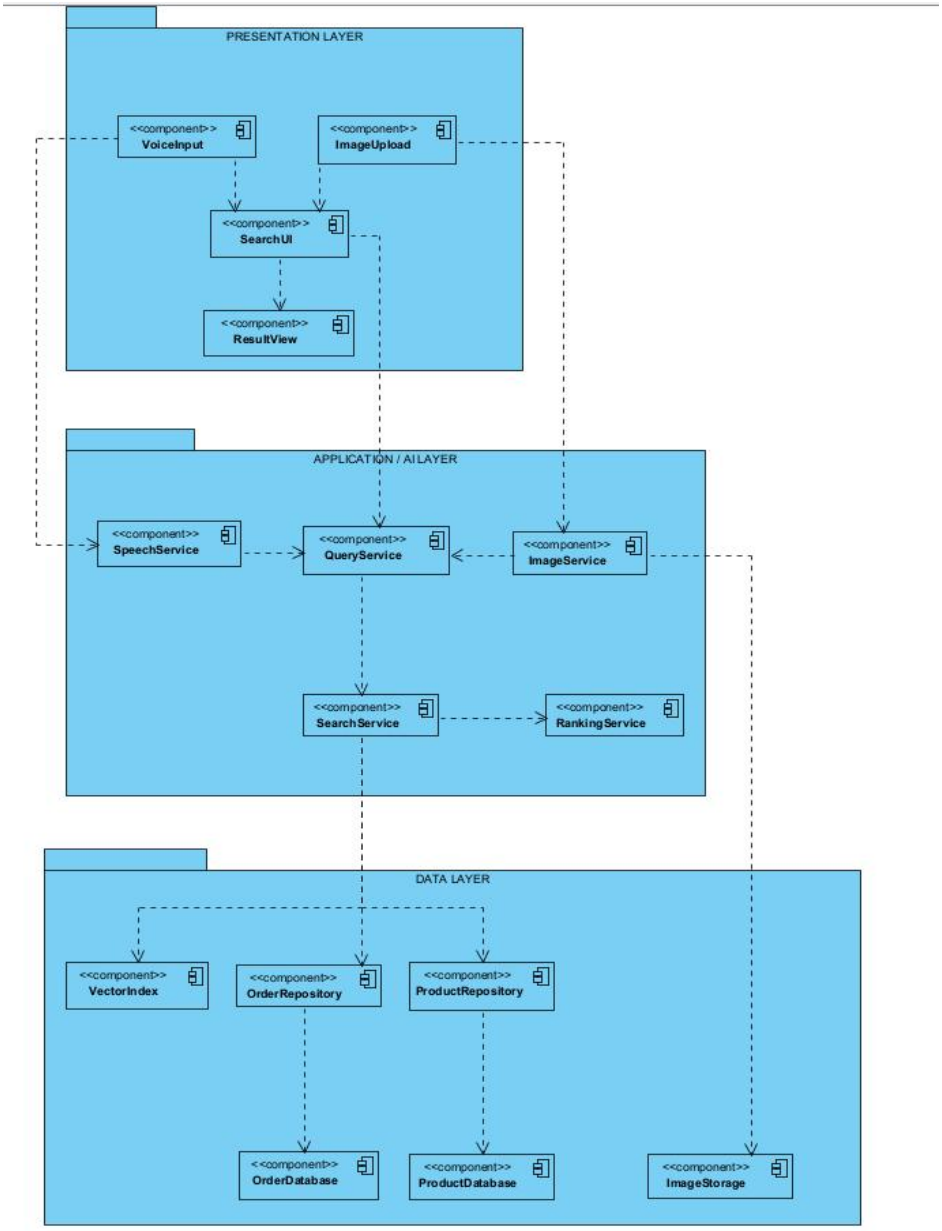


Figure 2 : three-layer architecture

3.2 Presentation Layer

The Presentation Layer collects input from the customer and displays search results. It contains no search logic and never accesses the database directly.

Component	Responsibility
SearchUI	Main entry point. Receives text queries, sends requests to the Application Layer, and passes the results to the result view.
VoiceInput	Captures the voice query (simulated by transcribed text) and forwards it to the SpeechService.
ImageUpload	Lets the customer take or choose a product photo and forwards it to the ImageService.
SearchResultView	Displays the ranked list of products or the order details, with scores.

3.3 Application / Intelligence Layer

The Application Layer contains the core logic of the system. It interprets queries, converts every input type into a common representation, retrieves candidates, and ranks them.

Component	Responsibility
QueryService	Converts text, voice, and image inputs into a common query object ({"type": ..., "query"/"embedding": ...}).
SpeechService	Converts voice input into text. In this prototype it is simulated: the transcribed text is passed through unchanged.
ImageService	Converts an uploaded image into a feature vector using an image encoder (artificial embeddings in this prototype).
SearchService	Retrieves candidate products by keyword matching or cosine similarity, and finds orders by ID.
RankingService	Orders the retrieved candidates by score, in descending order.

Retrieval (SearchService) and ranking (RankingService) are kept in separate components, because generating candidates and ordering them are different operations.

3.4 Data Layer

The Data Layer stores and provides access to all persistent information.

Component	Responsibility
ProductRepository	Provides access to product records (id, name, category, color, price, stock).
OrderRepository	Provides access to order records (order id, customer id, status, total).
VectorIndex	Stores product embeddings and supports similarity lookup.
ProductDatabase	Stores product data.
OrderDatabase	Stores order data.
ImageStorage	Stores product images.

The repositories hide the storage details. The Application Layer asks a repository for data and does not need to know how the data is stored.

4. Task 4: Sequence Diagram

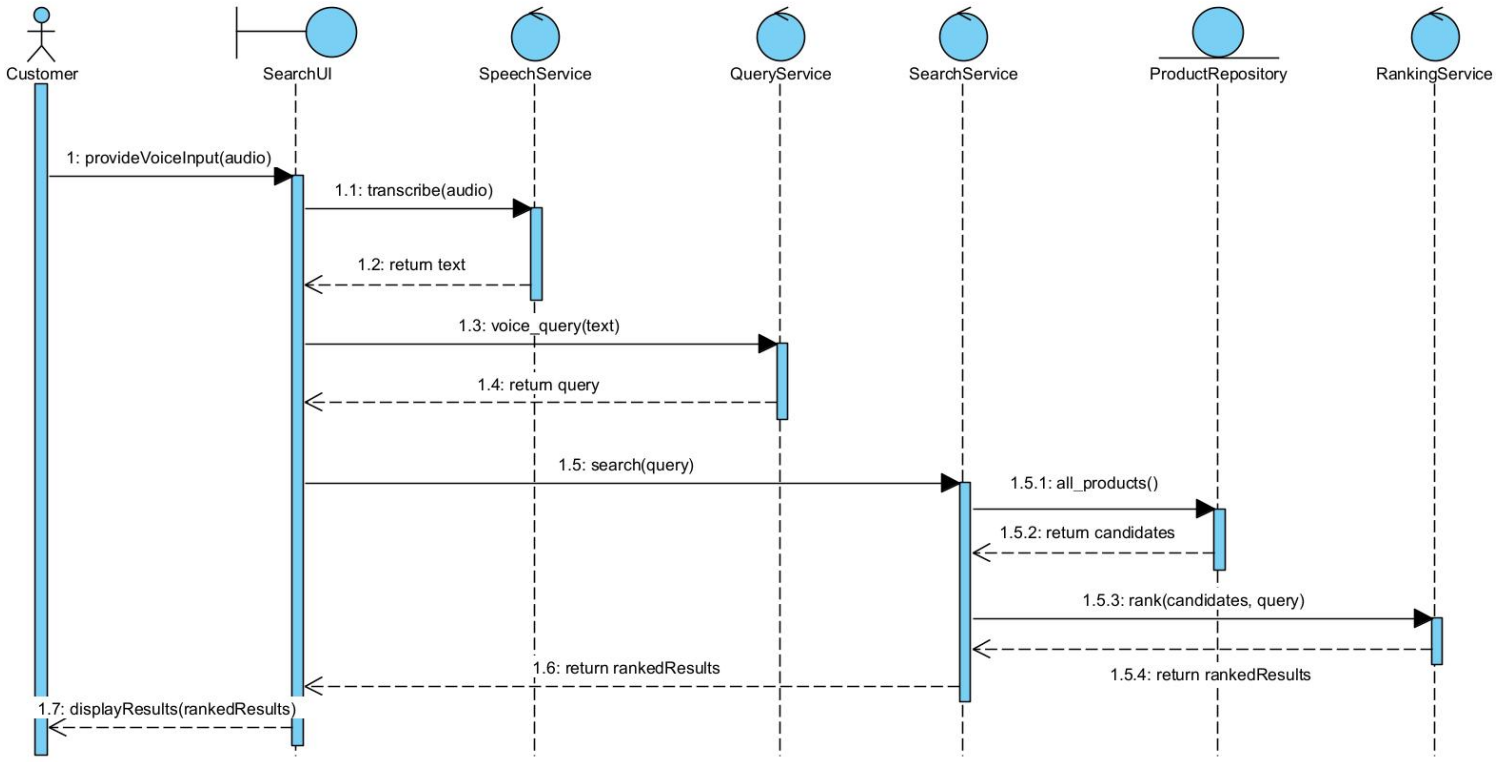


Figure 3 : Sequence Diagram

4.1 Overview

The sequence diagram shows how the system handles the scenario **"Customer searches for a product using voice."** It shows the order of messages between the customer and the system components, and how the request passes through the layers: Presentation → Application → Data.

4.2 Participants

Participant	Layer	Role
Customer	Actor	Provides the spoken query and receives the results.
SearchUI	Presentation	Receives the voice input, coordinates the calls, and displays the results.
SpeechService	Application	Converts audio into text (simulated in this prototype).
QueryService	Application	Builds the common query object from the transcribed text.

SearchService	Application	Retrieves candidate products for the query.
ProductRepository	Data	Provides access to the product records.
RankingService	Application	Orders the candidates by relevance score.

4.3 Message Sequence

#	Message	Description
1	Customer → SearchUI: provideVoiceInput(audio)	The customer speaks a query, for example "find black running shoes".
2	SearchUI → SpeechService: transcribe(audio)	The UI sends the audio for speech-to-text.
3	SpeechService → SearchUI: return text	The transcribed text is returned. In the prototype, the input text is passed through unchanged.
4	SearchUI → QueryService: voice_query(text)	The UI requests a query object.
5	QueryService → SearchUI: return query	The query is returned as {"type": "voice", "query": text}.
6	SearchUI → SearchService: search(query)	The UI sends the query to the search service.
7	SearchService → ProductRepository: all_products()	The search service requests the product records.
8	ProductRepository → SearchService: return candidates	The repository returns the products; keyword matching produces the candidate list.
9	SearchService → RankingService: rank(candidates, query)	The candidates are sent for ranking.

10	RankingService → SearchService: return rankedResults	The products are returned in descending order of score.
11	SearchService → SearchUI: return rankedResults	The ranked results go back to the UI.
12	SearchUI → Customer: displayResults(rankedResults)	The UI shows the products with their ranking scores.

5. Task 5 : Python Implementation

This project is a small local e-commerce product search prototype. It uses a Flask web application, a JSON product catalog, keyword-based ranking, and simulated voice input. Product photographs are displayed in the web catalog. The current implementation is a prototype and does not perform image-based retrieval.

5.1 Project Structure

```
assignment06/
|-- main.py                    Flask application entry point
|-- application/
|   |-- __init__.py
|   |-- query_service.py       Creates text, voice, and image query objects
|   |-- speech_service.py      Simulates transcription from supplied text
|   |-- search_service.py      Searches and ranks products by keywords
|-- data/
|   |-- __init__.py
|   |-- product_repository.py  Loads and provides product data
|   |-- products.json          Product catalog with 20 products
|-- templates/
|   |-- index.html             Product search page
|-- static/
|   |-- app.js                 Search-mode interaction
|   |-- style.css              Responsive page styling
|   |-- products/              Product photographs used by the page
|-- tests/
|   |-- test_multimodal_search.py
|-- requirements.txt
|-- README.md
|-- project_explanation.txt
```

5.2 Product Repository

The product catalog contains 20 sample products. Each product record includes an id, name, category, color, price, stock quantity, description, and image path. The product data is stored in data/products.json. ProductRepository loads the JSON file and provides the products to the application.

Product photographs are stored under static/products/. The image value in each JSON product record is a path relative to the static directory, for example:

```
"image": "products/large-black-umbrella.jpg"
```

The web page displays these photographs on product cards. The images are not converted into feature vectors and are not used for similarity search. The sample catalog does not currently include a brand field.

5.3 Main Services

Component	Responsibility
Flask application	Connects the services and serves the web page.
QueryService	Creates common query dictionaries for text, voice, or image input.
SpeechService	Returns the text supplied as simulated speech transcription; it does not access a microphone.
SearchService	Matches query terms against product name, category, color, and description, then sorts matching products by relevance score.
ProductRepository	Loads and provides products from products.json.
Presentation layer	templates/index.html displays the search page; static/app.js and static/style.css provide browser interaction and appearance.

SearchService uses a simple keyword score: a term in a product name adds 3 points; a term in the category or color adds 2 points; and a term found elsewhere in the searchable product text adds 1 point. Results are sorted from highest to lowest score. Ranking is part of SearchService; a separate RankingService is not present.

5.4 Implemented Features and Limitations

Implemented features:

- Local web interface for browsing and searching the product catalog.
- Keyword search using product names, categories, colors, and descriptions.
- Simulated voice-search mode: the user types the transcript that speech recognition would have produced.
- Display of product photographs stored in static/products/.
- Basic relevance scoring and result ordering.

Not implemented in this prototype:

- Real microphone or browser speech recognition.
- Image upload, image feature extraction, or image-similarity search.
- Vector embeddings or a persisted vector index.
- Multimodal query fusion.
- Synonym expansion, price/category filters, or stock-aware ranking.
- Order repository or order-status search.

5.5 Running the Application

Python 3.10 or later is recommended. Install the dependency and start the Flask application from the project directory:

```
python -m pip install -r requirements.txt
python main.py
```

Open the local web interface at:

```
http://127.0.0.1:5000
```

The project uses Flask, listed in requirements.txt. It is not a standard-library-only web application.

5.6 Demonstration

Keyword search example:

- Input: black umbrella
- Processing: QueryService creates a text query; SearchService matches and scores the product records.
- Expected top result: Large Black Umbrella
- Ranking score: 6 ("black" and "umbrella" each match the product name and add 3 points.)

Simulated voice example:

- Input transcript: brown wallet
- Processing: SpeechService returns the supplied transcript; QueryService marks it as a voice query; SearchService performs the same keyword search.
- Expected top result: Brown Leather Wallet
- Ranking score: 6 ("brown" and "wallet" each match the product name and add 3 points.)

The home page shows the full catalog and its product photographs. To produce execution evidence for a report, run the application, open the local URL, and capture screenshots of the product catalog and search results.

6. Task 6 : Demonstration

6.1 Web Demonstration

The keyword search filters by extracted price, the voice search identifies the corresponding smartphone, the image search ranks visually similar items, and the multimodal search integrates both text and visual evidence.

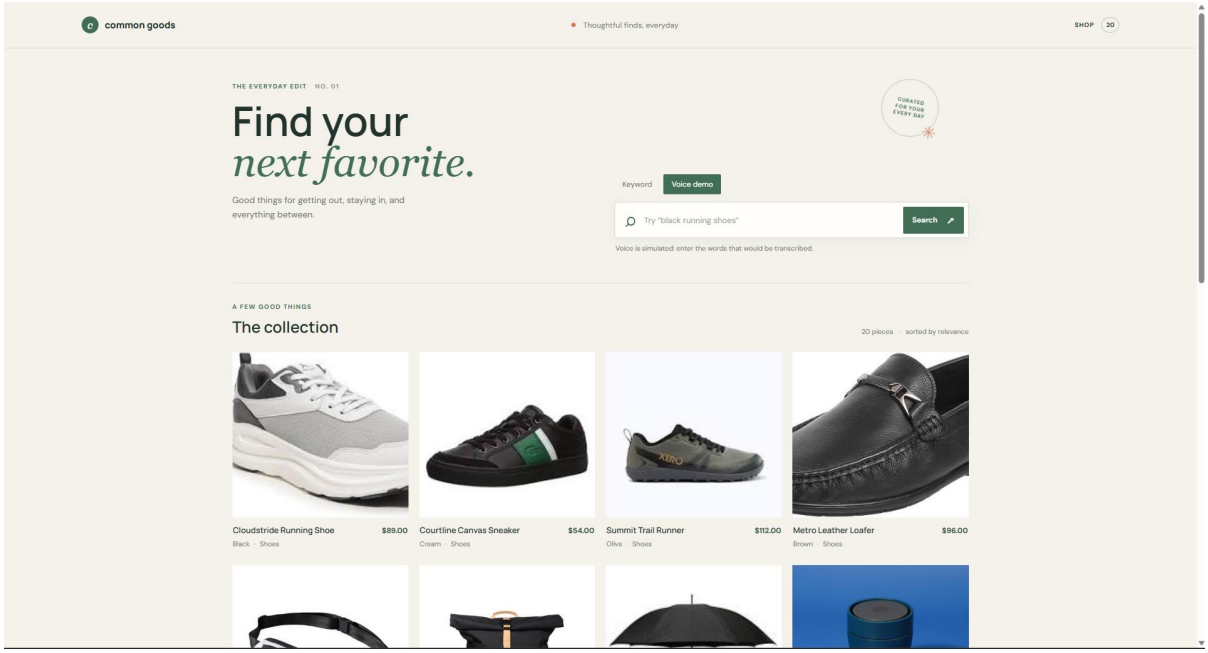


Figure 4 : Web page

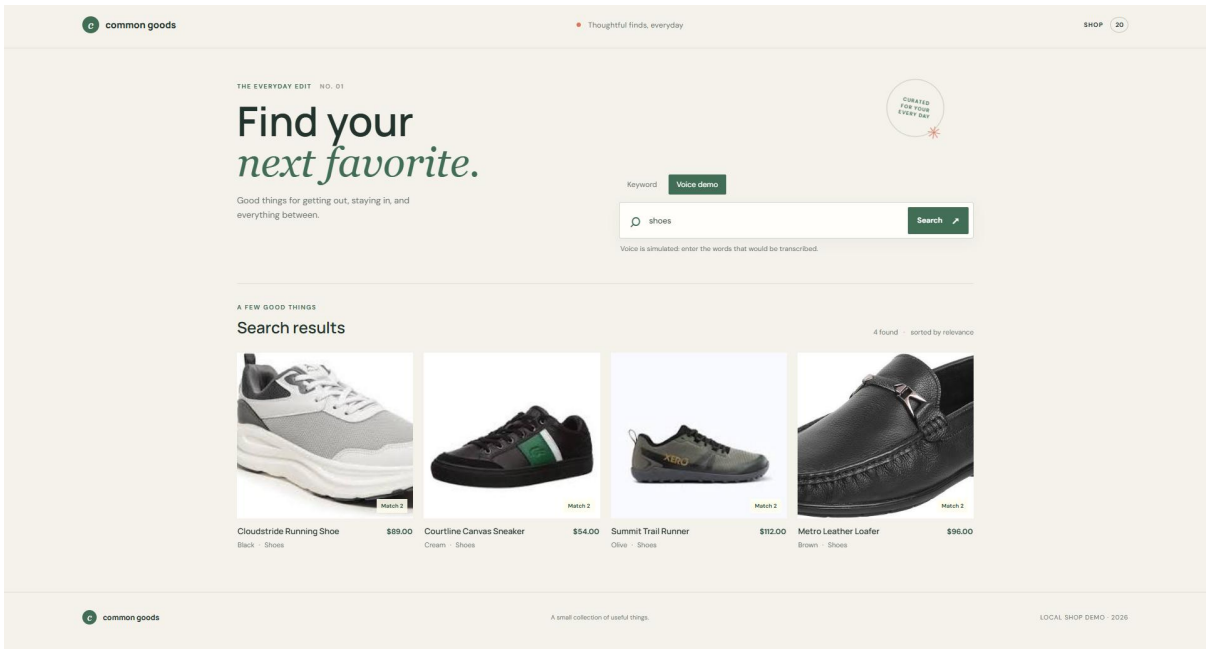


Figure 7: Web results for“shoes”

6.2 Test Set Evaluation

Ran six searches against the current catalog using the supported text and simulated-voice modes:

Query	Mode	Top-1 Result	Score	Success
black running shoes	Text	Cloudstride Running Shoe	7	Yes
brown leather wallet	Text	Brown Leather Wallet	9	Yes
large black umbrella	Text	Large Black Umbrella	9	Yes
find stainless steel travel mug	Voice	Stainless Steel Travel Mug	12	Yes
green water bottle	Voice	Green Water Bottle	9	Yes
white cotton bath towel	Voice	White Cotton Bath Towel	12	Yes

Successful: 6/6
Success rate: 100%
Incorrect results: None in this test set.

Link Git : https://github.com/khanhhsd/E-Commerce.github.io.git